Stream ciphers

- A stream cipher processes input elements continuously, producing output one element at a time.

- Stream ciphers process messages a bit or byte or character at a time when en/decrypting.
- Process the message bit by bit (as a stream)
- The most famous of these is the **Vernam cipher**
  - Invented by Vernam, working for AT&T, in 1917
  - Simply add bits of message to random key bits
  - Unconditionally secure provided key is truly random

- Typically have a random stream key
- Combined (XOR) with plaintext bit by bit
  - Ci = Mi XOR StreamKeyi

Stream ciphers

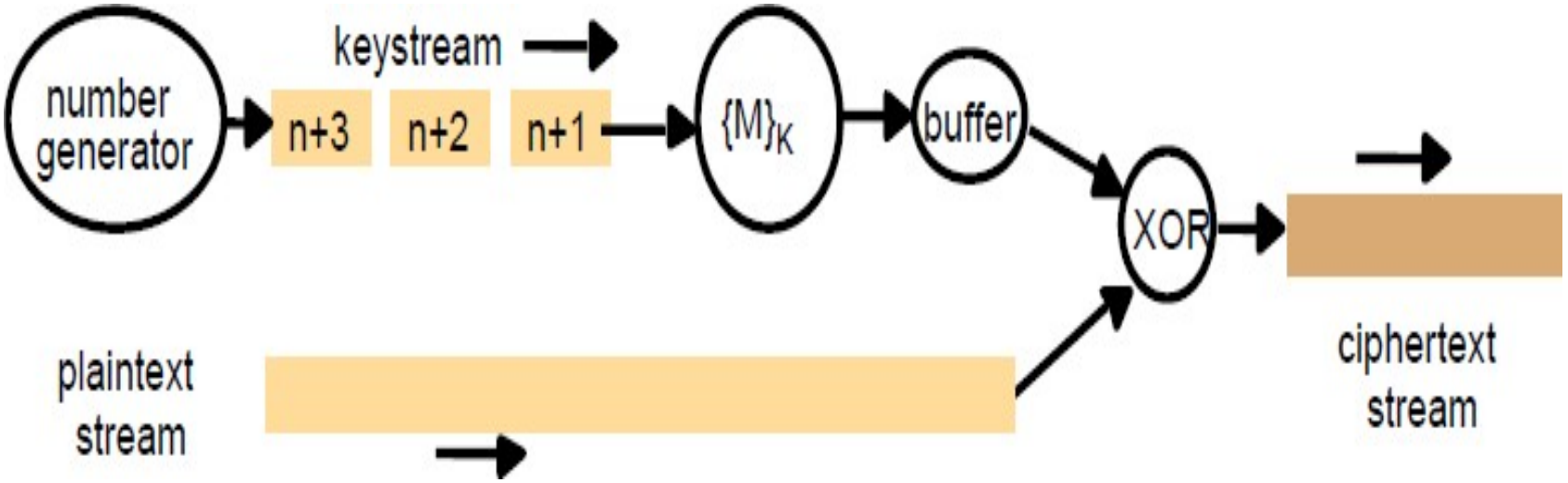
Continuous data streams, e.g., telephone samples of 8 bit length

– wasteful to fit into 64 bit block for encoding

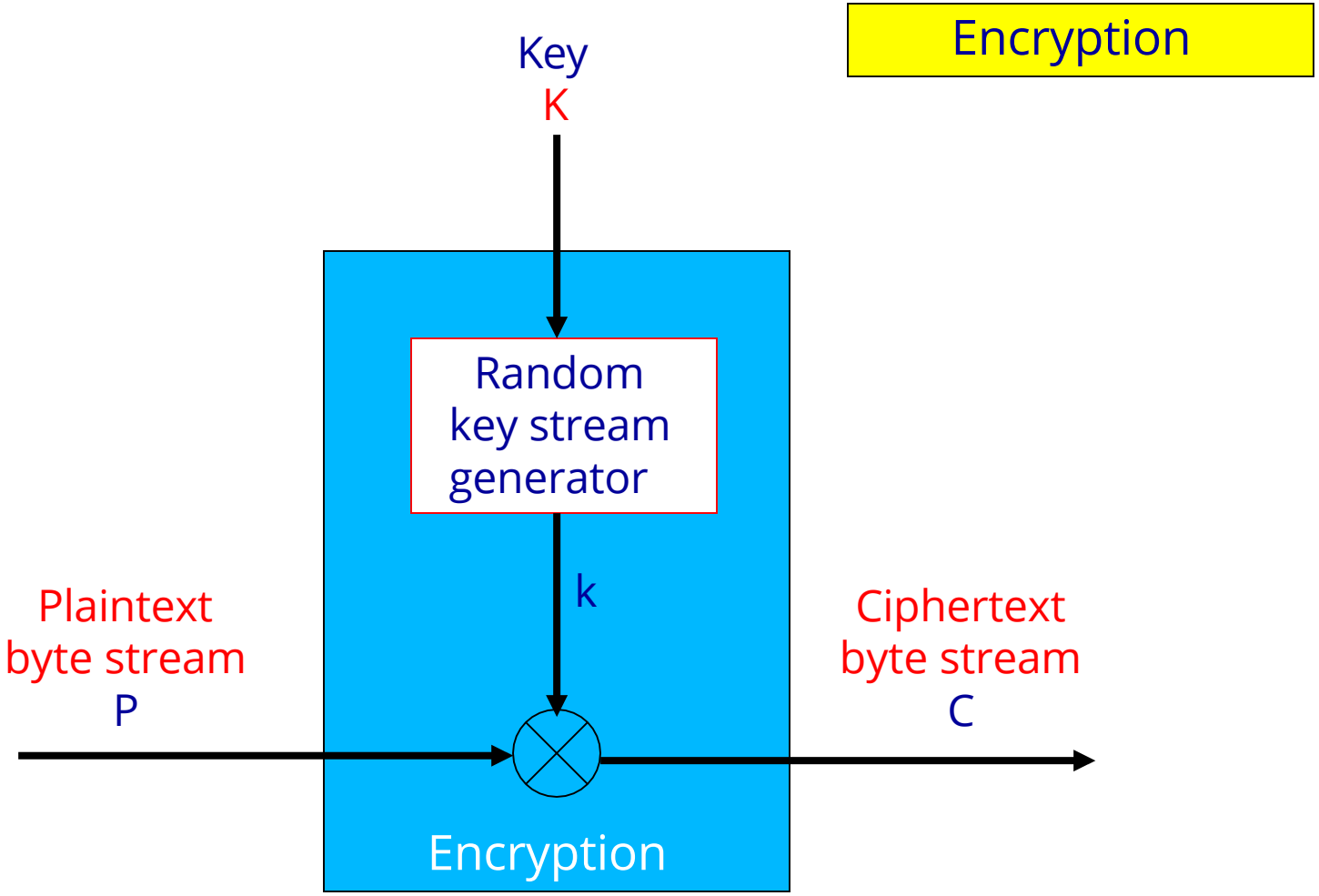
– works only for symmetric cryptography

– used when no block structure (e.g. voice)

Stream Cipher Structure

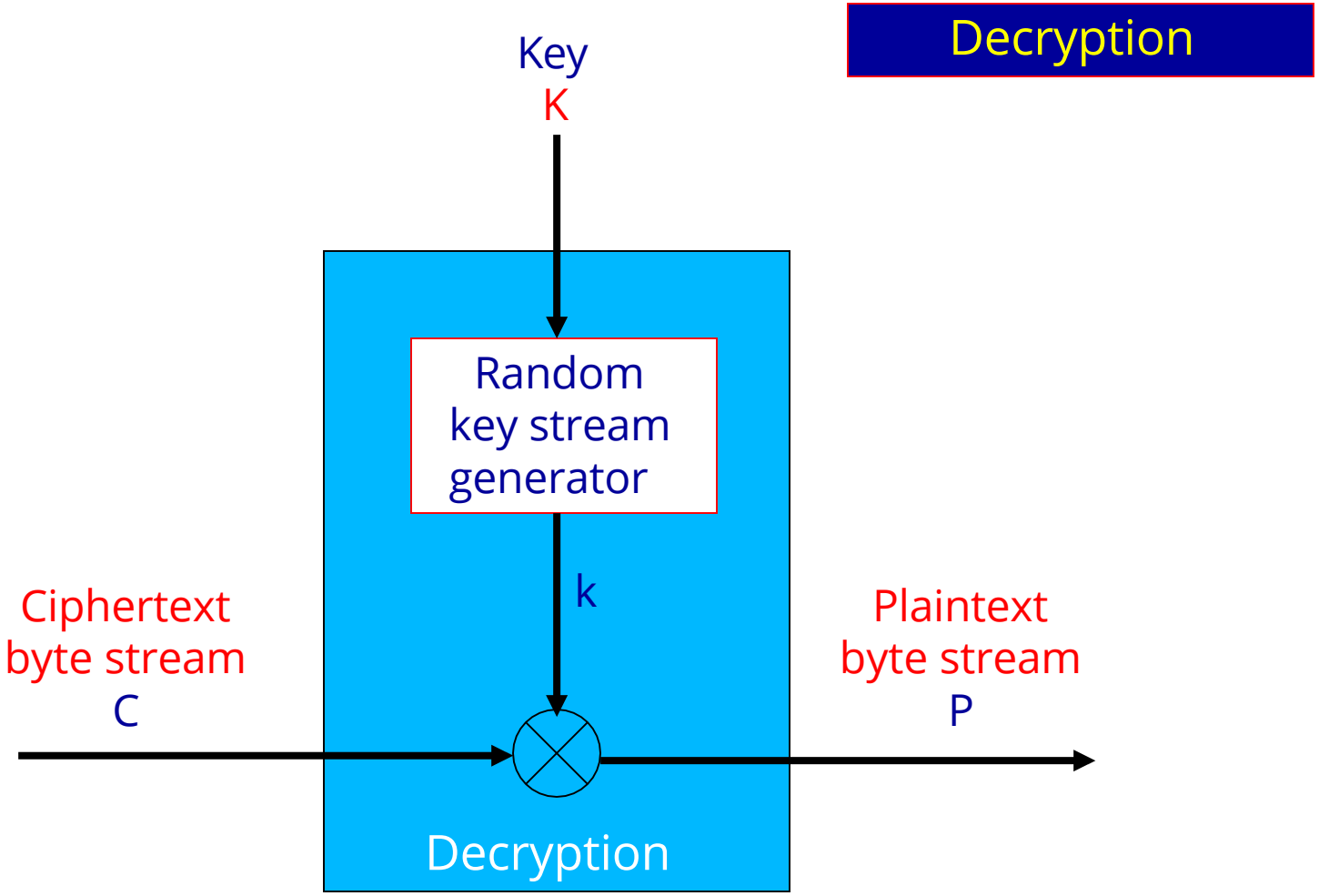


Stream Cipher Structure



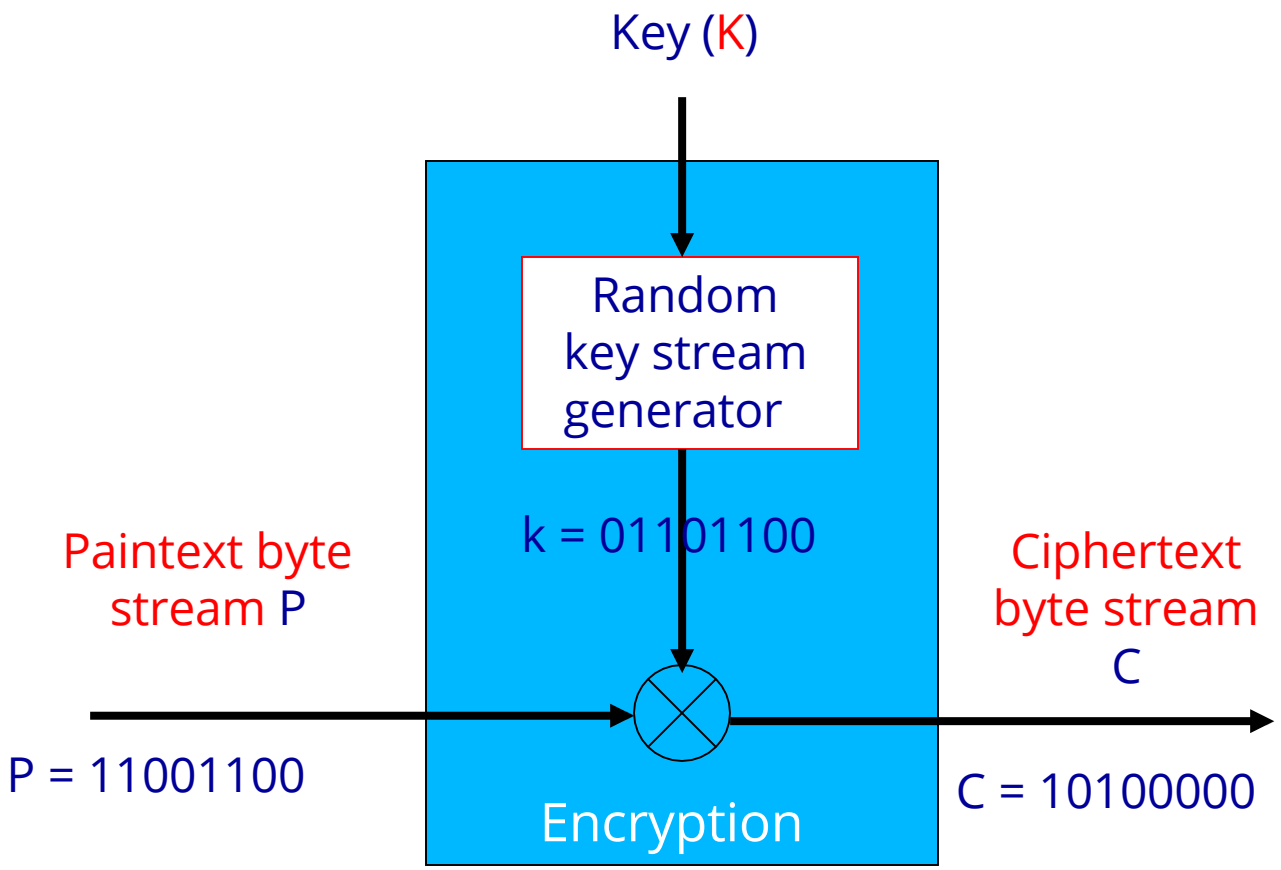
A stream cipher Encryption

Stream Cipher Structure



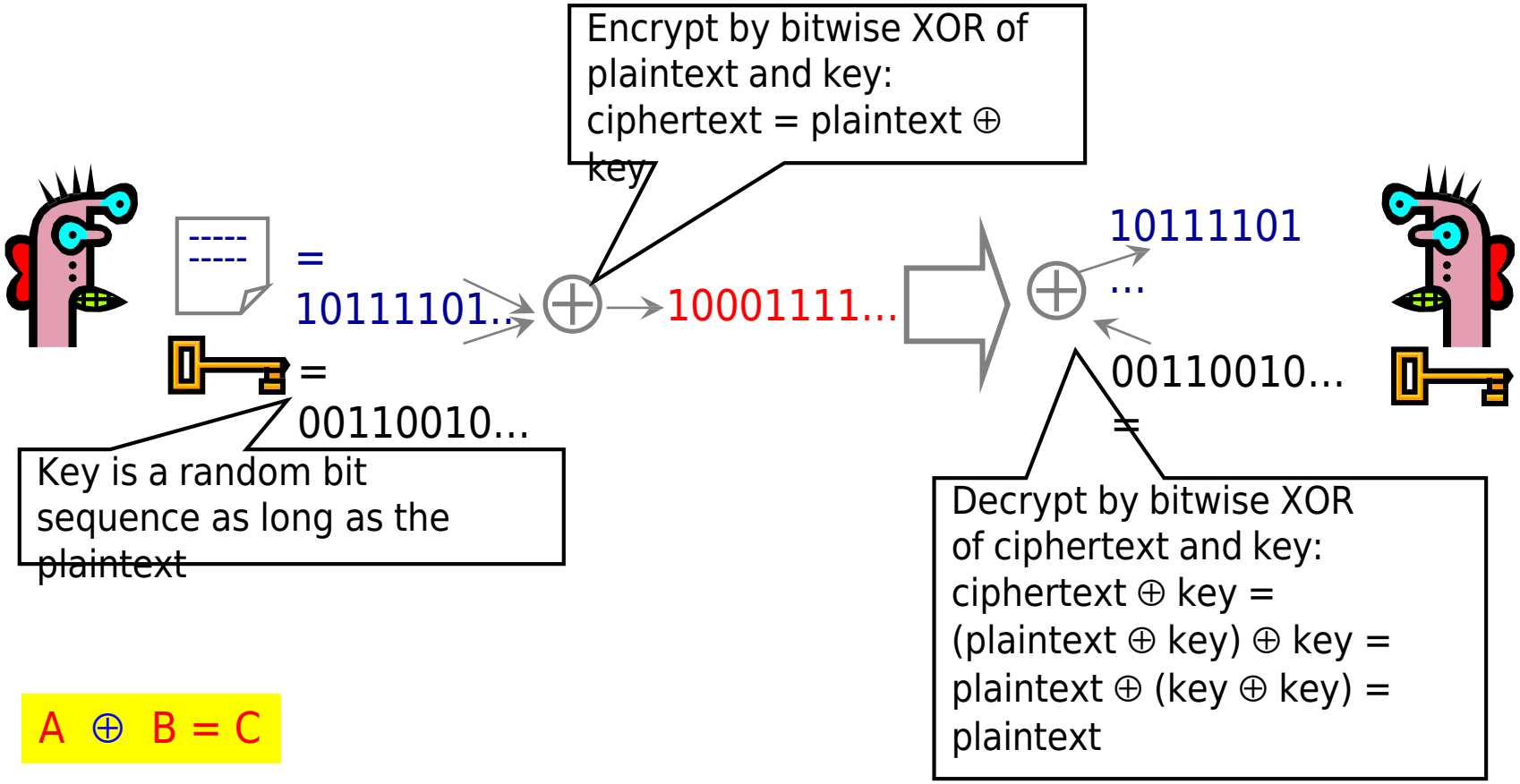
A stream cipher Decryption.

Stream Cipher Structure
Example



A stream cipher Encryption Example

Stream Cipher Structure



A ⊕ B = C

Here, ⊕ = XOR Symbol

C ⊕ B = A
C ⊕ A = B

RC4

- A stream cipher
- Designed in 1987 by Ron Rivest
- Variable key size
- Byte-oriented stream cipher
- Widely used in web and wireless
- 8 to 16 operations are required per output byte

- Extremely simple and easy to explain, very fast

RC4

- A variable-length key of from 1 to 256 bytes is used to initialise a 256-byte state array S, with elements S[0], S[1], ……, S[255].
- For encryption and decryption, a byte k is generated from S by selecting one of the 255 entries in a systematic way.
- As each value of key is generated, the entries in S are once again permuted

RC4 (3 Steps)

Step 1

Initialisation of S

- The entries of S are set equal to the values from 0 to 255 in ascending order. i.e.

S[0] = 0
S[1] = 1
……
……
……
S[255] = 255

Concept

RC4

Step 1

Initialisation of S

Code

//**

for i = 0 to 255 do

S[i] = i

//**

S[0] = 0

S[1] = 1

……

……

……

S[255] = 255

RC4

Given a key k of length m bytes

Step 2

Initial Permutation of S

// This involves starting with S[0] and going through S[255], and for each S[i], swapping S[i] with another byte in S as obtained by some mechanism

```
j = 0

for i = 0 to 255 do
    j = ( j + S[ i ] + k[ i mod m ] ) mod 256
    swap ( S[ i ], S[ j ] )
```

m = Size of key > 0

256 = Size of state array S

Result of mod 256 = 0 to 255

i mod m = 0 to m-1

RC4

Step 3

RC4 Encryption

- Encryption continues shuffling array values
- Sum of shuffled pair selects "stream key" value t
- XOR with next byte of message to en/decrypt

i = j = 0

for each message byte M_b

A → $i = (i + 1) \bmod 256$ ← **Increment in i**

B → $j = (j + S[i]) \bmod 256$ ← **Calculation of j**

C → $swap(S[i] , S[j])$ ← **Final Permutation of S**

D → $t = (S[i] + S[j]) \bmod 256$ ← **Calculation of t**

E → $Ci = M_b \text{ XOR } S[t]$ ← **XOR Operation**

RC4 Example

Given a key k of length **2 bytes**

k[0]	k[1]
10001001	01010001

S[0]	S[1]	S[2]	S[3]
00000000	00000001	00000010	00000011

Initialisation of S

//*****************************

for i = 0 to 3 do

S[i] = i

Loop executes according to the size of state array S

Initial permutation of S

j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

m = 2 = Size of Key

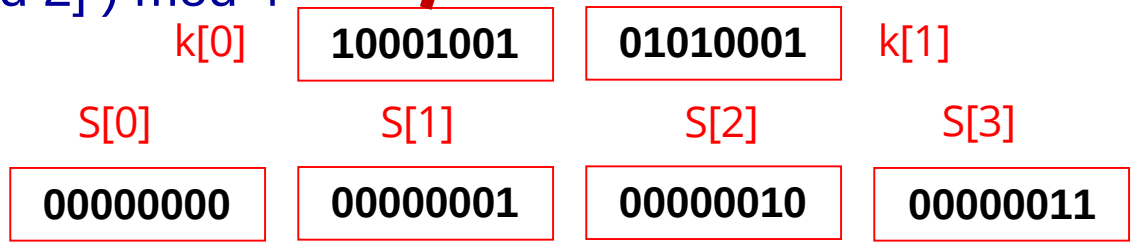
4 = Size of state array S

RC4 Example

Initial permutation of S

```
j = 0
for i = 0 to 3 do
    j = ( j + S[ i ] + k[ i mod 2] ) mod 4
    swap ( S[ i ], S[ j ] )
```

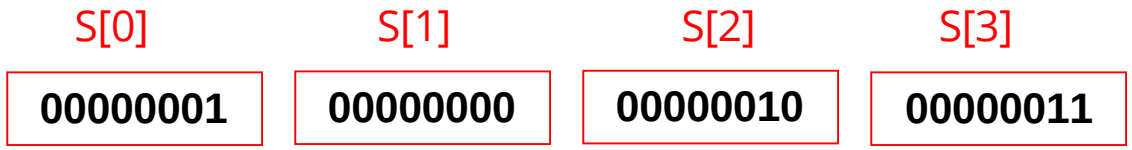
10001001 = 137 in Decimal



i=0

j = (0 + S[0] + k[0]) mod 4
= (0 +0+137) mod 4= 1

swap (S[0], S[1])



RC4 Example

Initial permutation of S

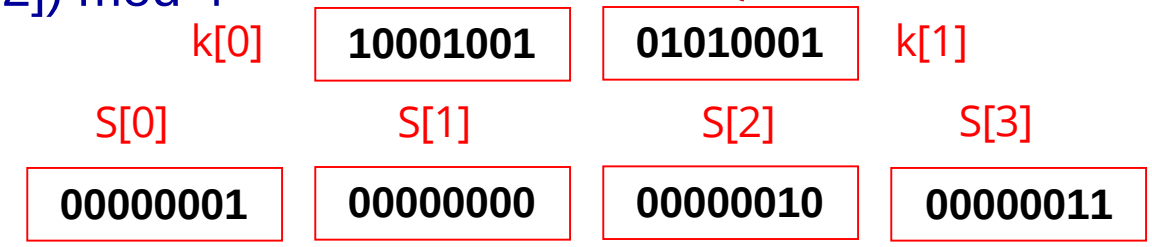
j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

01010001 = 81 in Decimal

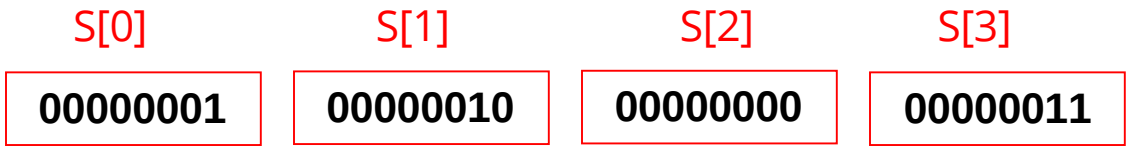


i=1

j = (1 + S[1] + k[1]) mod 4

= (1 + 0 + 81) mod 4= 2

swap (S[1], S[2])



RC4 Example

Initial permutation of S

j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

k[0]	10001001	01010001	k[1]

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000000	00000011

i=2

j = (2 + S[2] + k[0]) mod 4

= (2 +0+137)%4= 3

swap (S[2], S[3])

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000011	00000000

RC4 Example

Initial permutation of S

j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

k[0]	10001001	01010001	k[1]

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000011	00000000

i=3

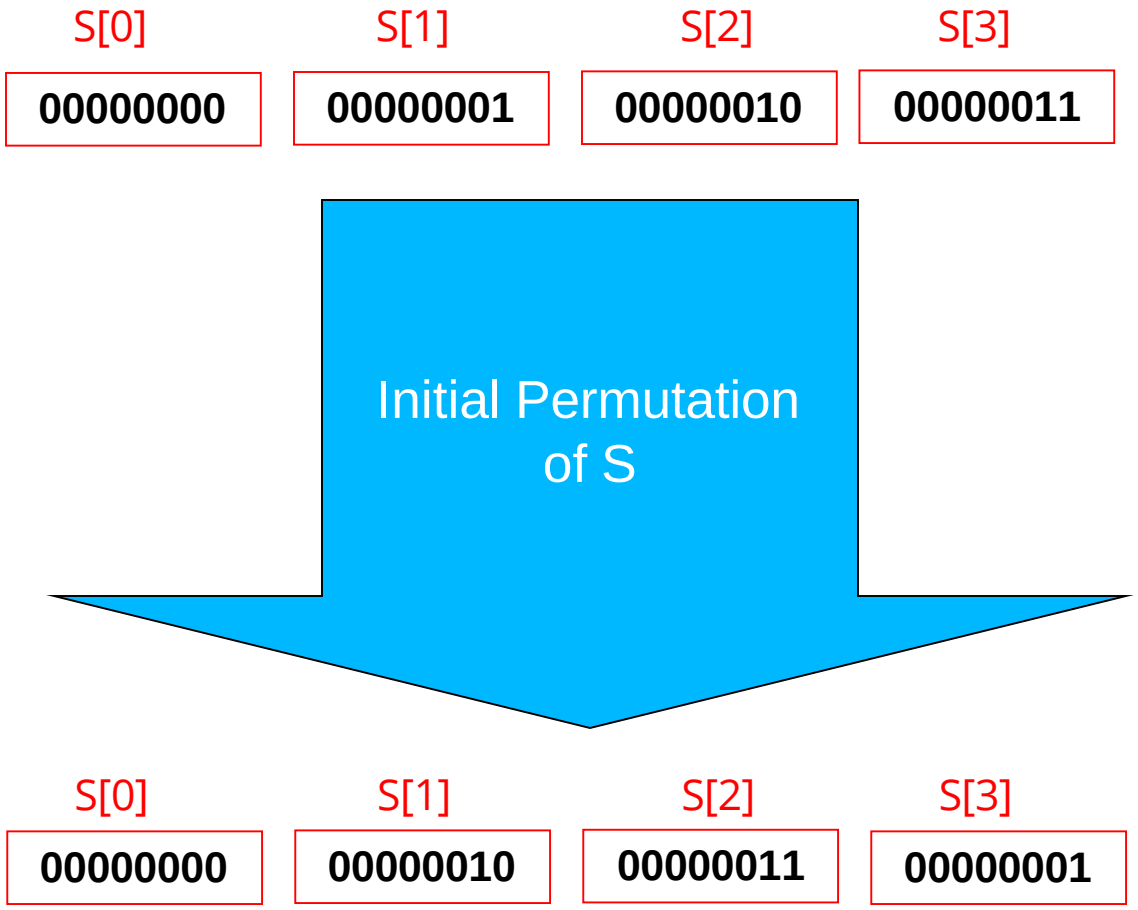
j = (3 + S[3] + k[1]) mod 4

= (3 +0+81)%4= 0

swap (S[3], S[0])

S[0]	S[1]	S[2]	S[3]
00000000	00000010	00000011	00000001

RC4 Example



Note: Not complete, Encryption Step still remaining.

RC4

1 **Initialisation of S**

S[0] = 0 , S[1] = 1 , ……, S[255] = 255

```
for i = 0 to 255 do
    S [ i ] = i
```

Given a key k of length m bytes

2 **Initial permutation of S**

```
j = 0
for i = 0 to 255 do
    j = ( j + S[ i ] + k[ i mod m ] ) mod 256
    swap ( S[ i ], S[ j ] )
```

3 RC4 Encryption

```
i = j = 0
for each message byte Mi
    i = ( i + 1 ) mod 256   , j = ( j + S[ i ] ) mod 256
    swap( S[ i ], S[ j ] )
    t = ( S[ i ] + S[ j ] ) mod 256
    Ci = Mi XOR S[ t ]
```

Final permutation of S

RC4 Example

Given a key k of length 2 bytes

k[0]	k[1]
10001001	01010001

1 Initialisation of S

```
for i = 0 to 3 do
    S[i] = i
```

S[0]	S[1]	S[2]	S[3]
00000000	00000001	00000010	00000011

2 Initial permutation of S

Plaintext:

10100010

```
j = 0
for i = 0 to 3 do
    j = ( j + S[ i ] + k [ i mod 2 ] ) mod 4
    swap ( S[ i ], S[ j ] )
```

i=0

```
j = ( 0 + S[0] + k[0] ) mod 4
  = ( 0 +0+137 ) % 4 = 1
swap ( S[0], S[1] )
```

S[0]	S[1]	S[2]	S[3]
00000001	00000000	00000010	00000011

RC4 Example

2

Initial permutation of S

j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

k[0]	10001001	01010001	k[1]

S[0]	S[1]	S[2]	S[3]
00000001	00000000	00000010	00000011

i=1

j = (1 + S[1] + k[1]) mod 4

= (1 +0+81) % 4 = 2

swap (S[1], S[2])

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000000	00000011

RC4 Example

2

Initial permutation of S

k[0]	10001001	01010001	k[1]

j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000000	00000011

i=2

j = (2 + S[2] + k[0]) mod 4

= (2 +0+137)%4= 3

swap (S[2], S[3])

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000011	00000000

RC4 Example

2

Initial permutation of S

k[0] | 10001001 | 01010001 | k[1]

j = 0

for i = 0 to 3 do

j = (j + S[i] + k[i mod 2]) mod 4

swap (S[i], S[j])

S[0]	S[1]	S[2]	S[3]
00000001	00000010	00000011	00000000

i=3

j = (3 + S[3] + k[1]) mod 4

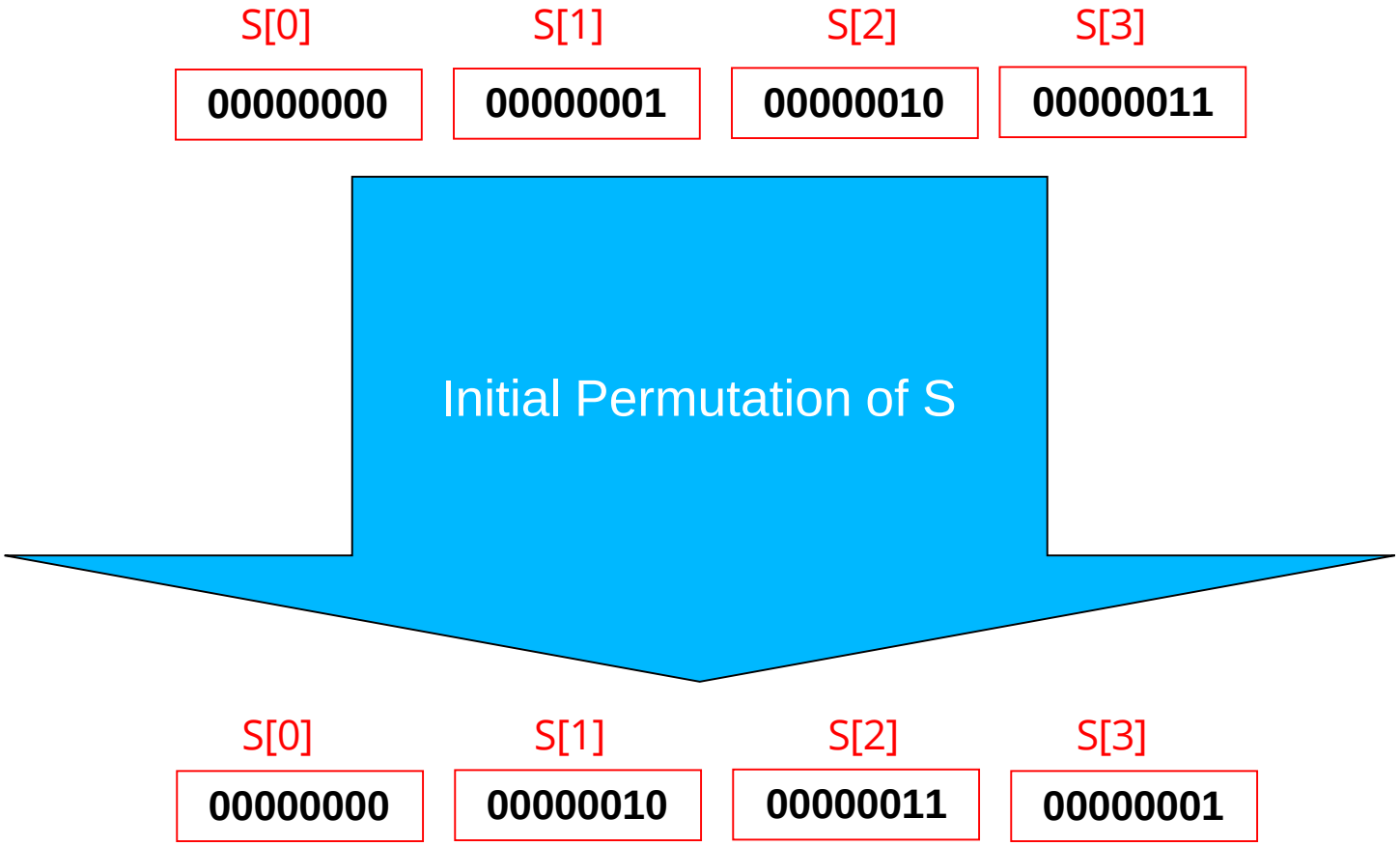
= (3 +0+81)%4= 0

swap (S[3], S[0])

S[0]	S[1]	S[2]	S[3]
00000000	00000010	00000011	00000001

RC4 Example

2



RC4 Example

k[0]	k[1]	S[0]	S[1]	S[2]	S[3]
10001001	01010001	00000000	00000010	00000011	00000001

RC4 Encryption:

i = j = 0

for each message byte M

i = (i + 1) mod 4 , j = (j + S[i]) mod 4

swap (S[i], S[j])

t = (S[i] + S[j]) mod 4

C = M XOR S[t]

Plaintext:

10100010
11001010

RC4 Encryption:

Plaintext:

1st Byte 10100010

i = j = 0

i = (i + 1) mod 4 = (0 + 1) mod 4 = 1

j = (j + S [i]) mod 4 = (0 + S [1]) mod 4 = (0 + 2) mod 4 = 2

swap (S[i], S[j]) = swap (S[1], S[2])

S[0]	S[1]	S[2]	S[3]
00000000	00000011	00000010	00000001

t = (S[i] + S[j]) mod 4 = (S[1] + S[2]) mod 4 = (3 + 2) mod 4 = 1

C = M XOR S[t] = 10100010 XOR S[1] = 10100010 XOR 00000011

= 10100001

RC4 Example

k[0]	k[1]	S[0]	S[1]	S[2]	S[3]
10001001	01010001	00000000	00000011	00000010	00000001

RC4 Encryption:

i = j = 0

for each message byte M

i = (i + 1) mod 4 , j = (j + S[i]) mod 4

swap (S[i], S[j])

t = (S[i] + S[j]) mod 4

C = M XOR S[t]

Plaintext:

10100010
11001010

RC4 Encryption:

i = 1, j = 2 See Previous Slide

Plaintext:

2nd Byte 11001010

i = (i + 1) mod 4 = (1 + 1) mod 4 = 2

j = (j + S [i]) mod 4 = (2 + S [2]) mod 4 = (2 + 2) mod 4 = 0

swap (S[i], S[j]) = swap (S[2], S[0])

S[0]	S[1]	S[2]	S[3]
00000010	00000011	00000000	00000001

t = (S[i] + S[j]) mod 4 = (S[2] + S[0]) mod 4 = (0 + 2) mod 4 = 2

C = M XOR S[t] = 11001010 XOR S[2] = 11001010 XOR 00000000

= 11001010

RC4 Practice Example

Calculate the Ciphertext for the following plaintext. Show the work step by step.

Given a key k of length m (2) bytes

k[0]	10001001	k[1]	01010001

Initialisation of S:

```
for i = 0 to 1 do
        S[i] = i
```

S[0]	00000000	S[1]	00000001

Initial permutation of S:

```
j = 0
for i = 0 to 1 do
        j = (j + S[i] + k[i mod m]) mod 2
        swap (S[i], S[j])
```

RC4 Encryption:

```
i = j = 0
for each message byte Mi
        i = (i + 1) mod 2
        j = (j + S[i]) mod 2
        swap(S[i], S[j])
        t = (S[i] + S[j]) mod 2
        Ci = Mi XOR S[t]
```

Plaintext:

10010001
11100101